A1-10 a page shared by two processes – step by step

every line of code, and the theory behind it

How to use this. Do one step at a time: read it, type the code yourself (do not copy-paste – typing is how it sticks), check the "Done when" box, then go on. Before you start: `task start a1-10`. At the end: `task test`, done when **9 of 9** checks pass and the final `exit` reports no memory leak.

From the list of earlier years: *simple shm* (shared memory). **Type S + E**: two syscalls, plus cleanup when a process ends (an event).

Theory: `concepts.pdf` §16 page tables, physical pages and shared pages; §4 page faults.

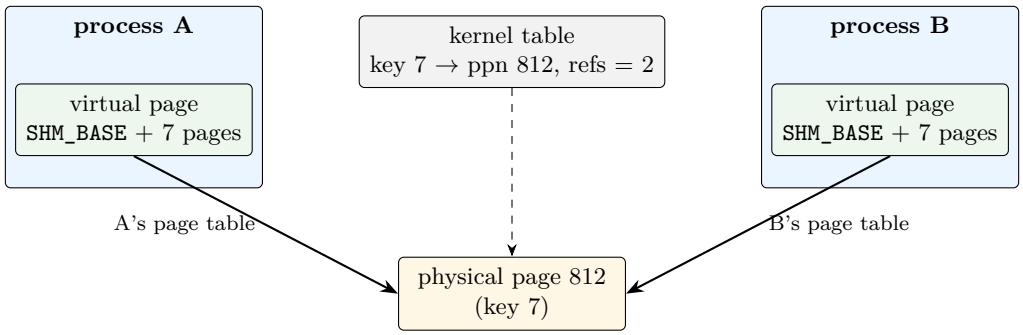
New to all of this? Read part 0 first.

Part 0 – the basics, in plain words

The words

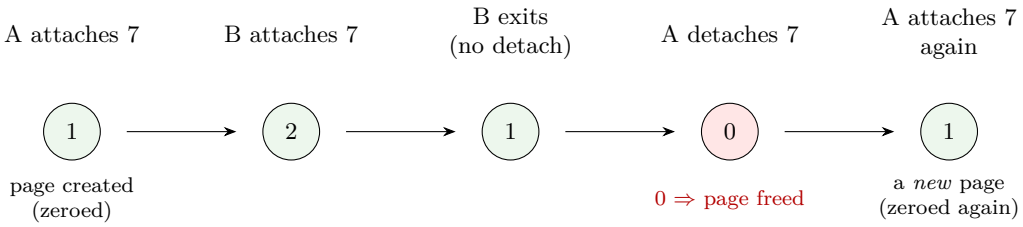
Word	What it means
Process	A running program with its own private memory. Normally two processes can *never* see each other's memory.
Address space	The addresses one process sees. Every process has its own; address `0x400000007000` in process A and in process B are normally different bytes.
Page	A 4096-byte piece of the address space. The kernel always works in whole pages.
Physical page (frame, ppn)	A 4096-byte piece of the real RAM, numbered ("physical page number").
Page table	The list per process: "my virtual page → this physical page".
Map / unmap	Enter / remove one line of that list (`mapPage`, `unmapPage`).
Shared memory	**One** physical page that appears in the page tables of **two** (or more) processes. Whatever one writes, the other reads – the fastest way for processes to talk.
Key	A number (here 1–15) that names a shared page. Two processes that both ask for key 7 get the *same* physical page.
Attach / detach	Attach = "put shared page k into my address space"; detach = "take it out again".
Reference count	How many processes currently have the page attached. Only when it drops to 0 may the page be given back to the RAM pool.
Free	Give a physical page back (`freePPN`), so the kernel can hand it to someone else.
Memory leak	A page that is never given back. SWEB checks at the end (`exit`) that no page leaked.
Kernel-wide table	One table for the whole kernel (not per process): key → physical page + reference count. Every process looks into the same table.
Global object / `startup()`	SWEB never runs the constructors of global C++ objects. A kernel-wide object is therefore created with `new` in `startup()`, the first kernel function.
Lock order / deadlock	Two threads that each hold one lock and wait for the other's lock wait forever (deadlock). The simple rule here: **never hold two locks at the same time**.
Destructor	The function that runs when an object is deleted – `~UserProcess` runs when a process ends.
TLB	The CPU's cache of "page → RAM" answers; after an unmap it must be flushed.

Picture 1 – one physical page, two processes



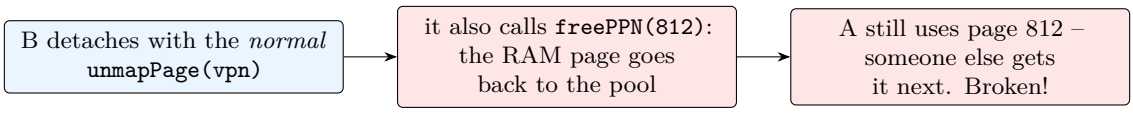
Both page tables point to the *same* physical page. A writes "hello" at its address, B reads "hello" at its address. The kernel-wide table remembers which physical page belongs to key 7 and how many processes use it.

Picture 2 – the reference count over time



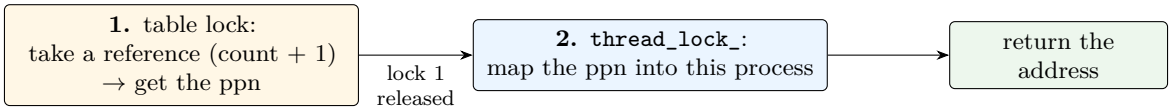
This is exactly the story of the test: the page must survive B's exit (count 1), and after A's detach (count 0) it is really gone – a new attach gets a fresh, empty page.

Picture 3 – why the normal `unmapPage` would break it



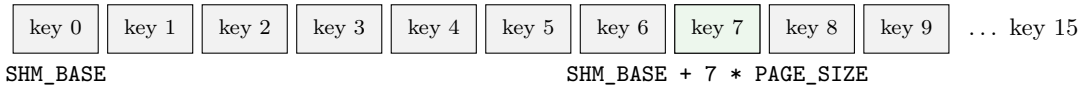
So you need an unmap that **does not free** (step 4). The same danger is hidden in `~ArchMemory`: when a process ends, it frees *every* mapped page – also the shared one. That is why the process must unmap its shared pages itself before its address space is deleted (step 11).

Picture 4 – two locks, never both at once



Attach: reference *first*, then map. Detach: unmap *first*, then drop the reference. Each lock is released before the other one is taken – so no deadlock is possible, and the page cannot be freed while we are mapping it (we already hold a reference).

Picture 5 – where the shared pages live



Key k always lands at the same address `SHM_BASE` + k pages (`SHM_BASE` = `0x400000000000`, far away from code, heap and stacks). Key 0 is not used.

The goal (level 1)

```
Name:         shm_attach, shm_detach
Nr:           1680, 1681
Description:  shm_attach(key) maps the shared page of key (1..15) into the calling
              process (at a fixed address per key) and returns its address. The first
              attach of a key creates the page (zeroed). Attaching twice in one process
              returns the same address.
              shm_detach(key) unmaps it. The page lives as long as at least one process
              has it attached - also if a process exits without detaching.
Return Value: attach: address or NULL; detach: 0 or -1 (not attached)
```

What each function does, in simple terms

Function	**What it does**
`shm_attach(key)` (libc)	Ask the kernel: "put shared page *key* into my memory, tell me where".
`shm_detach(key)` (libc)	Ask the kernel: "take shared page *key* out of my memory".
`SharedPages::acquire(key)`	In the kernel table: count + 1. If nobody had the page yet, get a fresh zeroed one first. Returns its ppn.
`SharedPages::release(key)`	Count − 1. If it reaches 0, give the physical page back.
`UserProcess::shmAttach(key)`	acquire, then map the page at the key's fixed address (only once per process).
`UserProcess::shmDetach(key)`	unmap *without freeing*, then release.
`unmapPage(vpn, false)`	Remove the page-table entry but *keep* the physical page.
`~UserProcess` (addition)	When a process ends: detach every key it still has.

The order: 11 steps

#	What	File	Why
1	two syscall numbers	`syscall-definitions.h`	kernel and libc agree
2	libc: declarations + wrappers	`nonstd.h`, `nonstd.c`	the program's entry
3	the two `case`s	`Syscall.cpp`	the kernel's entry
4	`unmapPage` that can keep the page	`ArchMemory.h/.cpp`	do not free a shared page
5	the `SharedPages` class	new `mm/SharedPages.h`	the kernel-wide table
6	its functions	new `mm/SharedPages.cpp`	acquire / release
7	create it at boot	`main.cpp`	no global constructors
8	declarations + bit array	`UserProcess.h`	per-process state
9	`shmAttach`	`UserProcess.cpp`	reference, then map
10	`shmDetach`	`UserProcess.cpp`	unmap, then release
11	cleanup at exit	`UserProcess.cpp`	the page survives / is freed correctly

1 Two syscall numbers — `common/include/kernel/syscall-definitions.h`

```
#define sc_shm_attach 1680
#define sc_shm_detach 1681
```

Both sides include this file, so the program and the kernel use the same numbers.

Done when

Two lines, next to the other `sc_....`

2 The libc part — `userspace/libc/include/nonstd.h` + `src/nonstd.c`

In `nonstd.h` (add `#include "types.h"` at the top if `size_t` is unknown there), before the closing `#ifdef __cplusplus`:

```
extern void* shm_attach(size_t key);
extern int shm_detach(size_t key);
```

At the end of `nonstd.c`:

```
void* shm_attach(size_t key)
{
  return (void*) __syscall(sc_shm_attach, key, 0x00, 0x00, 0x00, 0x00);
}

int shm_detach(size_t key)
{
  return __syscall(sc_shm_detach, key, 0x00, 0x00, 0x00, 0x00);
}
```

- The wrappers only enter the kernel. `shm_attach` returns an *address*, hence the `(void*)`; the kernel's 0 becomes `NULL`.
- `shm_detach` returns `int`: the kernel's `(size_t) -1` becomes `-1` – the test checks `shm_detach(7) == -1`.
- Declaration (`.h`) *and* definition (`.c`) – without the declaration the test does not compile.

Done when

The test compiles (it still fails: the kernel does nothing yet).

3 The two cases — `common/source/kernel/Syscall.cpp`

In the `switch` of `syscallException`, e.g. after `case sc_execv`:

```
case sc_shm_attach:
  return_value = currentThread->getProcess()->shmAttach(arg1);
  break;
case sc_shm_detach:
  return_value = currentThread->getProcess()->shmDetach(arg1);
  break;
```

The attached keys belong to the *process*, so the work goes straight to two `UserProcess` methods (step 9, 10) – no extra `Syscall::` functions needed.

Done when

Two cases.

4 An unmap that can keep the page `arch/x86/64/include/ArchMemory.h` + `source/ArchMemory.cpp`

In `ArchMemory.h` change the declaration:

```
bool unmapPage(uint64 virtual_page, bool free_physical_page = true);
```

In `ArchMemory.cpp` change the definition's first line and the `freePPN` line:

```
bool ArchMemory::unmapPage(uint64 virtual_page, bool free_physical_page)
{
  ...
  m.pt[m.pti].present = 0;
  if (free_physical_page)
    PageManager::instance()->freePPN(m.page_ppn);
  ...
```

- Picture 3: the old `unmapPage` *always* frees the physical page – fatal for a shared page.
- `= true` is a **default value**: every existing call `unmapPage(vpn)` keeps working exactly as before (frees). Only the shared-page code passes `false`.
- The default goes into the **declaration** only (`.h`), not into the definition (`.cpp`).

Done when

`unmapPage(vpn, false)` removes the entry and keeps the physical page.

5 The kernel-wide table

new file `common/include/mm/SharedPages.h`

```
#pragma once

#include "types.h"

class Mutex;

#define SHM_MAX_KEYS 16
#define SHM_BASE 0x0004000000000000ULL

class SharedPages
{
  public:
    static SharedPages* instance();
    static void init();
    size_t acquire(size_t key);
    void release(size_t key);

  private:
    SharedPages();
    static SharedPages* instance_;

    struct Entry
    {
      size_t ppn;
```

```
25        size_t references;
26      };
27      Mutex* lock_;
28      Entry entries_[SHM_MAX_KEYS];
29  };
```

Lines 7–8. 16 slots (keys 1–15 are valid, 0 is unused). `SHM_BASE`: the address of key 0; key k is at `SHM_BASE` + k pages (picture 5). Far away from code, heap (`0x6000...`) and stacks (`0x7FFF...`).

Lines 13–14. The usual SWEB "singleton" pattern (like `PageManager::instance()`): exactly one table in the kernel. `init()` creates it, `instance()` returns it.

Lines 15–16. The only two operations: count up (and maybe create), count down (and maybe free).

Lines 22–26. One entry per key: which physical page (`ppn`, 0 = none yet) and how many processes have it attached.

Line 27. A `Mutex*` (a pointer, created with `new` in the constructor): several processes can attach at the same time; the table must be changed by one at a time. `class Mutex;` in line 5 is enough for a pointer – the full header is only needed in the `.cpp`.

Done when

The header exists. (CMake picks up new files in `common/source/mm/` automatically; `task test` re-runs cmake.)

6 The table's functions

new file `common/source/mm/SharedPages.cpp`

```
1  #include "SharedPages.h"
2  #include "PageManager.h"
3  #include "Mutex.h"
4  #include "ScopeLock.h"
5  #include "assert.h"
6
7  SharedPages* SharedPages::instance_ = nullptr;
8
9  void SharedPages::init()
10 {
11   instance_ = new SharedPages();
12 }
13
14 SharedPages* SharedPages::instance()
15 {
16   assert(instance_ && "SharedPages::init() was not called in startup()");
17   return instance_;
18 }
19
20 SharedPages::SharedPages() : lock_(new Mutex("SharedPages::lock_"))
21 {
22   for (Entry& entry : entries_)
23   {
24     entry.ppn = 0;
25     entry.references = 0;
26   }
27 }
28
29 size_t SharedPages::acquire(size_t key)
```

```
30 {
31   ScopeLock lock(*lock_);
32   Entry& entry = entries_[key];
33   if (entry.ppn == 0)
34     entry.ppn = PageManager::instance()->allocPPN();
35   entry.references++;
36   return entry.ppn;
37 }
38
39 void SharedPages::release(size_t key)
40 {
41   ScopeLock lock(*lock_);
42   Entry& entry = entries_[key];
43   assert(entry.references > 0);
44   entry.references--;
45   if (entry.references == 0)
46   {
47     PageManager::instance()->freePPN(entry.ppn);
48     entry.ppn = 0;
49   }
50 }
```

Line 7. The static pointer starts as `nullptr` – a plain pointer needs no constructor, so this is fine.

Lines 9–12. `init` creates the one table with `new` (step 7 calls it at boot).

Lines 14–18. `instance` returns it; the `assert` catches a forgotten `init`.

Lines 20–27. Constructor: create the lock, mark every key as "no page, nobody".

Lines 29–37 – acquire. Under the table lock: if the key has no page yet, take a fresh physical page (`allocPPN` fills it with zeros – "the first attach creates the page (zeroed)"). Count + 1 and return the ppn.

Lines 39–50 – release. Count − 1. At 0 nobody uses the page any more: give it back (`freePPN`) and mark the key empty, so the next acquire creates a *new* page (picture 2).

Line 43. Releasing a key nobody holds is a bug in our own kernel code – the `assert` stops it at once.

Done when

`acquire` counts up (creating the page if needed), `release` counts down (freeing at 0).

7 Create the table at boot `common/source/kernel/main.cpp`

Add `#include "SharedPages.h"` at the top, and in `startup()` right after `Scheduler::instance();` (line 59):

```
SharedPages::init();
```

SWEB never runs constructors of global objects, so a global `SharedPages table;` would stay uninitialised. `startup()` runs once at boot, before any process exists – the right moment.

Done when

The table exists before the first process can attach.

8 Per-process state `common/include/kernel/UserProcess.h`

In the **public** part (e.g. after `cancelThread`):

```
size_t shmAttach(size_t key);
size_t shmDetach(size_t key);
```

At the very bottom of the **private** members:

```
bool shm_attached_[16] = {};
```

- `public`: `Syscall.cpp` calls them (a private method is "not accessible").
- `shm_attached_[k]` = "this process has key k mapped". Needed for three things: attaching twice gives the same address without a second mapping; detach of a key that is not attached returns -1; at exit we know which keys to clean up. `= {}` starts all as `false`.
- 16 = `SHM_MAX_KEYS`. Protected by `thread_lock_`, like the page table.

Done when

Two declarations (public) and one member (private).

9 shmAttach `common/source/kernel/UserProcess.cpp`

At the top add `#include "SharedPages.h"` (and `"ScopeLock.h"` if missing). The function goes at the end:

```
1  size_t UserProcess::shmAttach(size_t key)
2  {
3    if (key == 0 || key >= SHM_MAX_KEYS)
4      return 0;
5    size_t vpn = SHM_BASE / PAGE_SIZE + key;
6
7    size_t ppn = SharedPages::instance()->acquire(key);
8    bool already;
9    {
10     ScopeLock lock(thread_lock_);
11     already = shm_attached_[key];
12     if (!already)
13     {
14       bool mapped = arch_memory_->mapPage(vpn, ppn, 1);
15       assert(mapped && "the shm region is only mapped by shmAttach");
16       shm_attached_[key] = true;
17     }
18   }
19   if (already)
20     SharedPages::instance()->release(key);
21   return vpn * PAGE_SIZE;
22 }
```

Lines 3–4. Only keys 1–15. Error = 0, which the libc turns into `NULL` (the test checks `shm_attach(0)` and `shm_attach(16)`).

Line 5. The key's fixed page number (picture 5).

Line 7 – reference first. Count + 1 under the *table* lock (and released again inside `acquire`). From now on the page cannot be freed by anyone else.

Lines 9–18 – then map. An extra `{ }` block so the `ScopeLock` releases `thread_lock_` at its end (line 18), *before* line 20 might take the table lock again – never both locks at once (picture 4).

Lines 11–17. Not attached in this process yet: map the shared physical page at the key's address (1 = user may access it) and set the bit. `mapPage` returns `false` only if the page is already mapped – impossible here, the bit says it is not – hence the `assert`.

Lines 19–20. Attached already: we just took a *second* reference for the same process – give it back, one process counts once. The address is the same anyway.

Line 21. The address = page number × 4096.

Done when

Attach maps the page once per process and returns the same address every time.

10 `shmDetach` `common/source/kernel/UserProcess.cpp`

```
1  size_t UserProcess::shmDetach(size_t key)
2  {
3    if (key == 0 || key >= SHM_MAX_KEYS)
4      return (size_t) -1;
5    {
6      ScopeLock lock(thread_lock_);
7      if (!shm_attached_[key])
8        return (size_t) -1;
9      arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
10     ArchMemory::flushTlb();
11     shm_attached_[key] = false;
12   }
13   SharedPages::instance()->release(key);
14   return 0;
15 }
```

Lines 3–4, 7–8. Invalid key, or not attached → −1 ("detaching twice fails").

Line 9 – unmap, but keep. `false`: other processes may still use the physical page.

Line 10. The CPU may still remember the old translation – flush, or this process could keep writing into the page after detaching.

Line 11. Clear the bit.

Line 13 – then release. Outside `thread_lock_`. If we were the last user, the table frees the page now ("after the last detach the page was freed: a new one is zero").

Done when

Detach unmaps without freeing, then drops the reference.

11 Cleanup when a process ends `UserProcess.cpp, ~UserProcess`

In the destructor, between `delete loader_;` and `delete arch_memory_;` (line 33–34):

```
1  for (size_t key = 1; key < SHM_MAX_KEYS; ++key)
2  {
3    if (shm_attached_[key])
4    {
5      arch_memory_->unmapPage(SHM_BASE / PAGE_SIZE + key, false);
6      SharedPages::instance()->release(key);
7    }
8  }
```

Why here. A process may end *without* `shm_detach` (the test's second child does). Then `delete arch_memory_` would run `~ArchMemory`, which frees *every* mapped page – including the shared one the parent still uses (picture 3).

Line 5. So unmap our shared pages first, *without* freeing. Afterwards `~ArchMemory` does not see them any more.

Line 6. Drop our reference – otherwise the count never reaches 0 and the page leaks (the final `exit` of the test checks for leaks).

No lock, no TLB flush. The process is being deleted: none of its threads runs any more, and its page table is never loaded again.

Must come before `delete arch_memory_`. After that line the page table is gone.

Done when

`task test`: **9 of 9 passed**, no leak reported at the end.

The test – what each check means

`shm_basic.sweb` attaches key 7, writes a text, then starts `shm_child.sweb` twice.

Check	Which step makes it pass
attached key 7	steps 1–3, 5–9
attaching twice gives the same address	step 9: the bit, lines 19–20
child sees the parent's text (child 1)	one physical page in two page tables (picture 1)
the child wrote into our page	the same, in the other direction
page survives a child that did not detach (child 2)	step 11 + step 4: unmapped without freeing, count 2 → 1
detached	step 10
detaching twice fails	step 10, the bit check
after the last detach the page was freed: a new one is zero	step 6 `release` frees at 0; `acquire` makes a new zeroed page
invalid keys rejected	step 9, lines 3–4
no leak at the final `exit`	step 11: the parent ends attached; its reference is dropped

Say it in three sentences

Remember

- Shared memory = **one physical page in several page tables**; a kernel-wide table maps key → page and **counts** the users.
- Unmapping a shared page must **not free** it – only the last user (count 0) frees it, also when a process ends without detaching.
- Attach: reference, *then* map. Detach: unmap, *then* release. **Never hold both locks**.

Questions a tutor might ask

Question	Short answer
How can two processes see the same memory?	both page tables point to the same physical page
Why a reference count?	the page may only be freed when the last process lets go
Why not the normal `unmapPage`?	it frees the physical page – the other process would lose it
What happens if a process exits without detaching?	`~UserProcess` unmaps without freeing and releases, before `~ArchMemory` would free it
Why is the table created in `startup()`?	SWEB does not run global constructors
Why reference first, then map?	while mapping, the page must not be freed by another process's release
Why never both locks at once?	two processes taking them in opposite order would deadlock
Why the same address per key?	simple, and pointers inside the page mean the same in every process
Why `flushTlb` in detach?	otherwise the CPU might still translate the old address
Attach twice in one process?	second reference is dropped again; the bit prevents a second mapping
What would fork do with an attached page?	your team's fork copies the address space – the child would get a *copy*, not the shared page (a good follow-up question)